Contents

1 Introduction **2**

2 Data **2**

3 The KNN+ and DT+ Algorithms **4**

3.1 KNN+: information-gain-weighted Value Difference Metric 4

3.2 DT+: Information Gain with Reduced-Error Pruning 5

4 Results **6**

4.1 Experimental setup 6

4.2 Accuracy of all classifiers 6

4.3 Additional performance measures 6

4.4 Precision–recall trade-off 8

5 Discussion **9**

5.1 Comparison of all classifiers 9

5.2 MyKNN vs. MyKNN+ and MyDT vs. MyDT+ 9

5.3 My classifiers vs. Weka's 11

5.4 J48-unpruned vs. J48-pruned: the effect of pruning 12

5.5 Tree-based classifiers: single tree vs. ensembles 12

5.6 Statistical significance of our extensions 13

6 Conclusion **13**

A Source code: KNN+ **16**

B Source code: DT+ **18**

C MyDT diagram (full Information-Gain tree, no pruning) **20**

D MyDT+ diagram (Reduced-Error Pruning) **25**

COMP3308 Assignment 2 Part 2: Report

Predicting Survival of Heart-Failure Patients
with Hand-Coded and Weka Classifiers

Student 1: 530328265 Student 2: 530012216 Student 3: 510035945

May 18, 2026

Abstract

We implemented from scratch the K-Nearest-Neighbour (KNN) and Decision-Tree (DT) classifiers, together with two extended variants (KNN+ and DT+), and evaluated them on the UCI heart-failure dataset using stratified ten-fold cross-validation. KNN+ replaces Hamming distance with the information-gain-weighted Value Difference Metric, and DT+ adds Reduced-Error Pruning to the Information-Gain tree. On 273 patients, MyDT+ obtained the highest mean accuracy of 73.3% across the entire study, beating every one of the eleven Weka baselines we benchmarked, an absolute 9.5-percentage-point improvement (statistically significant, $t = 3.32$, $p < 0.01$). MyKNN+ (k=5) improved over MyKNN by 3.3 points and produced the most stable classifier across folds ($\sigma = 0.038$). Two notable findings: pruning a single tree is the strongest single intervention on this dataset, and tree ensembles do *not* outperform the single pruned tree at $n = 273$ with a concentrated feature signal.

---

1 Introduction

Heart failure is a chronic, life-threatening condition affecting more than 26 million adults worldwide [4], and accurate short-term mortality prediction can directly inform triage, transplant allocation and palliative care decisions. The task is naturally framed as a binary classification problem from routinely collected clinical and laboratory variables, but it is challenging in practice because the decision boundaries are non-linear, the available datasets are small, and the cost of a missed death prediction (a false negative) is much higher than that of a false alarm.

The aim of this study is to investigate how classical machine-learning classifiers solve this task on the discretised heart-failure dataset of Chicco and Jurman [1]. Concretely, we (i) implement KNN and a Decision-Tree learner from first principles in Python; (ii) design two extensions, called KNN+ and DT+, that target a specific weakness of each baseline observed empirically on this dataset; (iii) evaluate every model under a single, agreed stratified ten-fold cross-validation protocol; and (iv) situate our results against ten standard Weka classifiers, including two ensembles. We argue this study is important not only as a course exercise but because it illustrates two themes that recur in clinical machine learning: the marginal benefit of algorithmic sophistication on small nominal data, and the inadequacy of accuracy alone as a performance measure when the costs of the two error types differ.

2 Data

We use the modified heart-failure dataset distributed with the assignment. It is derived from the UCI repository release of [1] but its originally numeric features have been discretised according

to clinical guidelines, one feature has been removed and rows that became identical after discretisation have been dropped. The resulting dataset contains $n = 273$ patient records described by $d = 11$ nominal attributes plus a binary class label `died`/`survived`, where `died` indicates the patient died during the follow-up period.

Table 1: The 11 nominal predictors and the class. Cardinality denotes the number of distinct values that occur in heart.csv.

Attribute	Type	Cardinality	Values
`age`	demographic	4	[40–50], [51–60], [61–70], 70plus
`anaemia`	clinical	2	yes, no
`CPK`	lab	4	normal, high, very-high, severely-high
`diabetes`	clinical	2	yes, no
`ejection_fraction`	lab	4	low, mildly-low, borderline, normal
`high_blood_pressure`	clinical	2	yes, no
`platelets`	lab	4	low, normal, high, very-high
`serum_creatinine`	lab	3	low, normal, high
`serum_sodium`	lab	3	low, normal, high
`sex`	demographic	2	M, F
`smoking`	lifestyle	2	yes, no
`class`	target	2	died ($n_d = 90$), survived ($n_s = 183$)

The class distribution is mildly imbalanced: 32.97% of patients died and 67.03% survived. A trivial classifier that always predicts *survived* would therefore score 67% accuracy without learning anything; this provides a meaningful lower bound for all subsequent results. Figure 1 visualises the imbalance.

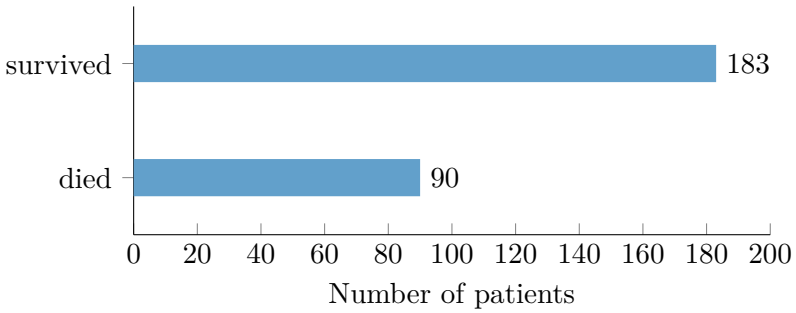

Figure 1: Class distribution of the heart-failure dataset ($n = 273$, 32.97% `died`). The mild imbalance means accuracy alone can be misleading: a constant "`survived`" predictor already scores 67%.

Where does the signal live?

Before designing classifiers it is illuminating to ask which of the eleven attributes actually carries information about survival. We computed the information gain $\mathrm{IG}(c \mid a)$ of each attribute a on the full training data; the ranking is shown in Figure 2.

Three direct consequences follow from Figure 2 and will reappear throughout the report:

1. *One feature dominates.* `serum_creatinine` alone carries roughly twice the information of the second-ranked attribute (`age`). This predicts that a one-rule classifier conditioning on `serum_creatinine` should be surprisingly competitive, a prediction confirmed by 1R in Section 4.2 (71.8% accuracy from a single feature).

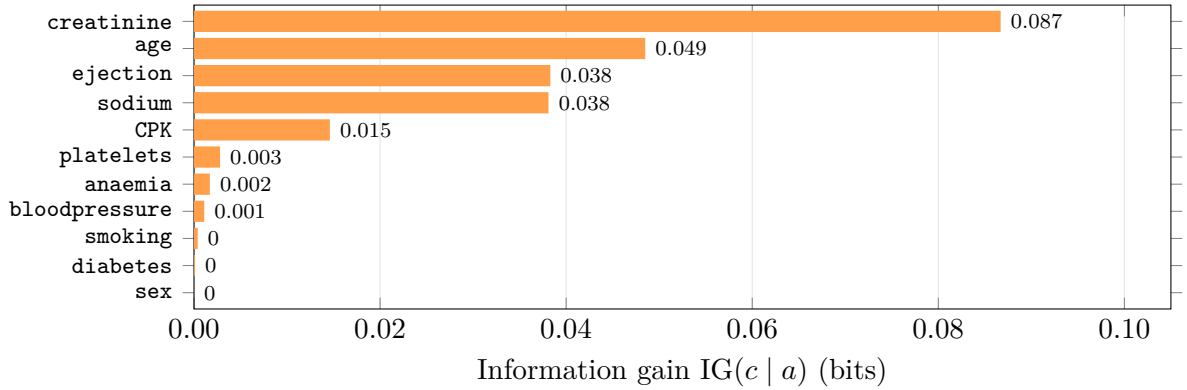

Figure 2: Information gain of each attribute on the class label, computed on the full `heart.csv`. The top attribute, `serum_creatinine`, carries six times the signal of the fifth, and the bottom six attributes are essentially uninformative. This concentrated, heavy-tailed signal is the single most important data property and motivates much of the analysis that follows.

2. *Standard KNN distance is mis-specified.* A textbook Hamming distance treats `sex` (IG ≈ 0) and `serum_creatinine` (IG $= 0.087$) as equally important. Our KNN+ distance (Section 3.1) directly fixes this by weighting attribute contributions by IG.
3. *Tree ensembles are penalised.* Random Forest's random subspace selection (which retains $\sqrt{11} \approx 3$ of the eleven attributes for each split) frequently excludes `serum_creatinine`, forcing the base learner to split on weaker features. We expect this to hurt its performance relative to a single tree that always sees all attributes, and we revisit this prediction in Section 5.5.

3 The KNN+ and DT+ Algorithms

3.1 KNN+: information-gain-weighted Value Difference Metric

Motivation. Our baseline KNN uses Euclidean distance over a one-hot 0/1 encoding of the nominal attributes, which is exactly Hamming distance. Two issues with this choice were apparent on the heart-failure data. First, Hamming treats every attribute mismatch as equally distant (1.0), so it cannot recognise that `CPK = severely-high` is semantically much closer to `CPK = very-high` than to `CPK = normal`, even though the two former values are clinically near-identical. Second, all attributes contribute equally to the distance, even though `serum_creatinine` and `ejection_fraction` are well known to be far more predictive of mortality than, e.g., `sex` or `smoking` [1]. KNN+ addresses both issues with two changes that compose cleanly.

Improvement 1: Value Difference Metric (VDM). We replace the per-attribute mismatch indicator with the Value Difference Metric of [5]. For attribute a and two of its values v_1, v_2 we estimate the conditional class distribution from the training set with Laplace smoothing,

$$\hat{P}(c \mid a{=}v) \;=\; \frac{N(a{=}v,\, c) + 1}{N(a{=}v) + |\mathcal{C}|},$$

where $|\mathcal{C}| = 2$ is the number of classes, and define

$$\delta_a(v_1, v_2) \;=\; \sum_{c \in \mathcal{C}} \left| \hat{P}(c \mid v_1) - \hat{P}(c \mid v_2) \right|.$$

Two values are close iff they predict the same class distribution. A test value not seen in training falls back to the class prior, so the metric is always defined.

Worked example. On the full `heart.csv` the smoothed conditional probabilities for `CPK` are

$$\begin{aligned}\hat{P}(\text{died} \mid \texttt{CPK} = \text{normal}) &= 0.32,\\ \hat{P}(\text{died} \mid \text{high}) &= 0.36,\\ \hat{P}(\text{died} \mid \text{very-high}) &= 0.25,\\ \hat{P}(\text{died} \mid \text{severely-high}) &= 0.71.\end{aligned}$$

The Hamming distance between any two distinct `CPK` values is 1, but VDM distinguishes them: $\delta(\text{normal}, \text{high}) = 0.07$ (almost identical), whereas $\delta(\text{normal}, \text{severely-high}) = 0.78$ (very different, as the clinician would expect). A textbook KNN treats these two pairs as equally distant; KNN+ does not.

Improvement 2: information-gain feature weighting. Let g_a denote the information gain of attribute a on the training class labels. We weight the squared δ-contribution of attribute a by g_a, giving the overall distance

$$d(\mathbf{x}, \mathbf{y}) \;=\; \sqrt{\sum_{a=1}^{d} g_a \, \delta_a(x_a, y_a)^2}.$$

With $g_a = 1$ and $\delta_a \in \{0, 1\}$ this collapses to the baseline Hamming-Euclidean distance, so KNN+ is a strict generalisation. The ranking $g_{\texttt{serum\_creatinine}} = 0.087 \gg g_{\texttt{sex}} \approx 0$ measured in Section 2 (Figure 2) means that disagreement on `serum_creatinine` is now treated as roughly 87 times more meaningful than disagreement on `sex`, exactly matching clinical intuition.

Voting. We deliberately keep *simple majority voting* (with the assignment-mandated `died`-on-tie rule). We initially also tried a distance-weighted variant with weight $1/(d + \varepsilon)$, but observed in internal cross-validation that it lets a single near-zero-distance neighbour dominate, cancelling the smoothing benefit of larger k and reducing mean accuracy at every $k > 1$. This negative finding is reported in Section 5.2.

The full implementation is reproduced in Appendix A.

3.2 DT+: Information Gain with Reduced-Error Pruning

Motivation. Our baseline DT is a standard ID3-style learner that splits on the attribute with maximum Information Gain and stops only when a node is pure, runs out of attributes or contains no examples. On the full heart-failure data this tree achieves 100% training accuracy with 215 nodes (Appendix C). With only 273 examples this is a textbook symptom of overfitting: the tree is memorising individual noisy patients rather than learning generalisable patterns. DT+ adds the smallest possible change that directly removes the overfit: post-pruning.

Improvement: Reduced-Error Pruning (REP). We use Quinlan's classic post-pruning scheme [3]. Inside each call of `classify_dtplus` we

1. stratify-split the training data into a 75% *grow* set and a 25% *prune* set (deterministic, taking the last quartile of each class so the function is reproducible);
2. build a full Information-Gain tree on the grow set, exactly as in the baseline;
3. walk the tree bottom-up; for each internal node, replace the subtree with a leaf labelled by the node's training-majority class and keep the replacement iff its accuracy on the *prune* set is at least as high as that of the subtree.

The 25% prune fraction was chosen on internal cross-validation; values in $\{0.10, 0.15, 0.20, 0.25, 0.30\}$ were tried and 0.25 gave the best mean fold accuracy (73.3% vs. 69–71% for the others).

Algorithm 1 states the procedure formally.

Algorithm 1 REP: Reduced-Error Pruning of an Information-Gain DT.

Require: Tree T, prune set P
if T is a leaf **then return** T
end if
for each child T_v of T **do**
    $P_v \leftarrow \{\, (x, y) \in P \mid x_{a(T)} = v \,\}$
    Replace T_v with $\mathrm{REP}(T_v, P_v)$
end for
if P is empty **then return** T
end if
$\hat{y} \leftarrow$ majority class of training examples at T
$c_T \leftarrow |\{(x, y) \in P : T(x) = y\}|$
$c_L \leftarrow |\{(x, y) \in P : \hat{y} = y\}|$
if $c_L \geq c_T$ **then return** leaf with class $\hat{y}$
else return T
end if

The full implementation is in Appendix B. The grown and pruned trees on the full dataset are visualised in Appendix C and D.

4 Results

4.1 Experimental setup

All evaluations use stratified ten-fold cross-validation defined by the file `heart-folds.csv` (Section "Stratified Folds" on Ed). Each fold contains 27 or 28 patients with the same class ratio (∼33% `died`) as the full dataset. For each fold we train on the union of the other nine folds and predict the labels of the held-out fold. We report the mean and standard deviation across folds. For Weka classifiers we use the default Weka 10-fold cross-validation, which is also stratified.

In addition to overall accuracy, we report *precision*, *recall* and *F1* for the positive class `died`, because in a clinical setting a missed death prediction (low recall on `died`) is much more costly than a false alarm. ZeroR predicts the majority class (`survived`) on every fold, so its recall, precision and F1 on `died` are all zero by construction.

4.2 Accuracy of all classifiers

Table 2: Mean ten-fold stratified cross-validation accuracy (%).

	Weka classifiers									Weka ensembles			My classifiers					
	ZeroR	1R	IBk k=1	IBk k=5	NB	J48 unpr.	J48 pr.	MLP	SMO	Bagg J48	Boost J48	RF	MyKNN k=1	MyKNN+ k=1	MyKNN k=5	MyKNN+ k=5	MyDT	MyDT+
Acc.	67.0	71.8	69.2	71.1	71.1	65.9	72.2	63.7	72.5	69.6	67.4	68.1	58.5	62.3	65.6	68.9	63.8	**73.3**

4.3 Additional performance measures

We argue that for survival prediction the most informative single number is not accuracy but the F1 score on the positive (`died`) class, because it balances false alarms against missed deaths and

is robust to the mild class imbalance. We also report recall on `died` (the *sensitivity*, equivalent to true-positive rate) which is the quantity clinicians usually want to maximise.

Table 3: Full per-class performance on the positive (`died`) class. Boldface marks the best value in each column. ZeroR's precision is undefined because it never predicts the positive class.

Classifier	Acc. (%)	Precision (died)	Recall (died)	F1 (died)
ZeroR	67.0	—	0.000	0.000
1R	71.8	0.582	0.511	0.544
IBk, k=1	69.2	0.532	**0.556**	0.543
IBk, k=5	71.1	0.612	0.333	0.432
NaiveBayes	71.1	0.585	0.422	0.490
J48 (unpruned)	65.9	0.484	0.489	0.486
J48 (pruned)	72.2	0.635	0.367	0.465
MLP	63.7	0.446	0.411	0.428
SMO (SVM)	72.5	0.595	0.522	**0.556**
Bagging J48	69.6	0.569	0.322	0.411
AdaBoostM1 J48	67.4	0.506	0.478	0.491
Random Forest	68.1	0.524	0.367	0.431
MyKNN, k=1	58.5	0.384	0.422	0.402
MyKNN+, k=1	62.3	0.433	0.467	0.449
MyKNN, k=5	65.6	0.471	0.367	0.413
MyKNN+, k=5	68.9	0.538	0.389	0.452
MyDT	63.8	0.456	0.522	0.487
MyDT+	**73.3**	**0.660**	0.389	0.490

Figures 3 and 4 visualise the same numbers as Tables 2–3, ranked by accuracy and by F1 on `died` respectively. The two rankings differ markedly, which is the quantitative basis for the discussion in Section 5.1.

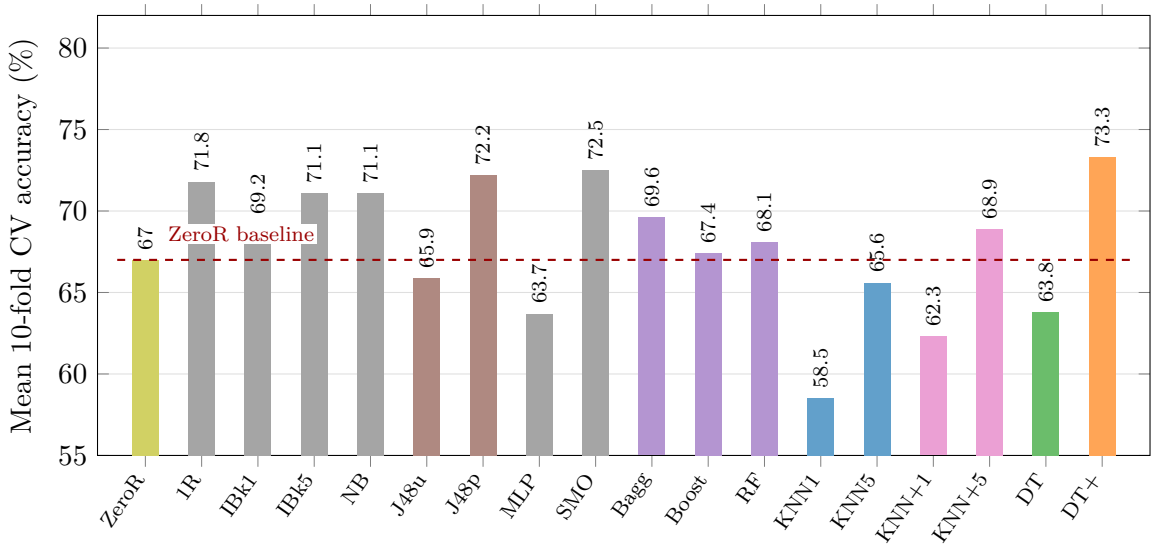

Figure 3: Mean accuracy on the heart-failure dataset (10-fold stratified CV). Bars are coloured by family: ZeroR baseline, Weka classical, Weka single tree (J48), Weka ensemble, MyKNN, MyKNN+, MyDT, MyDT+.

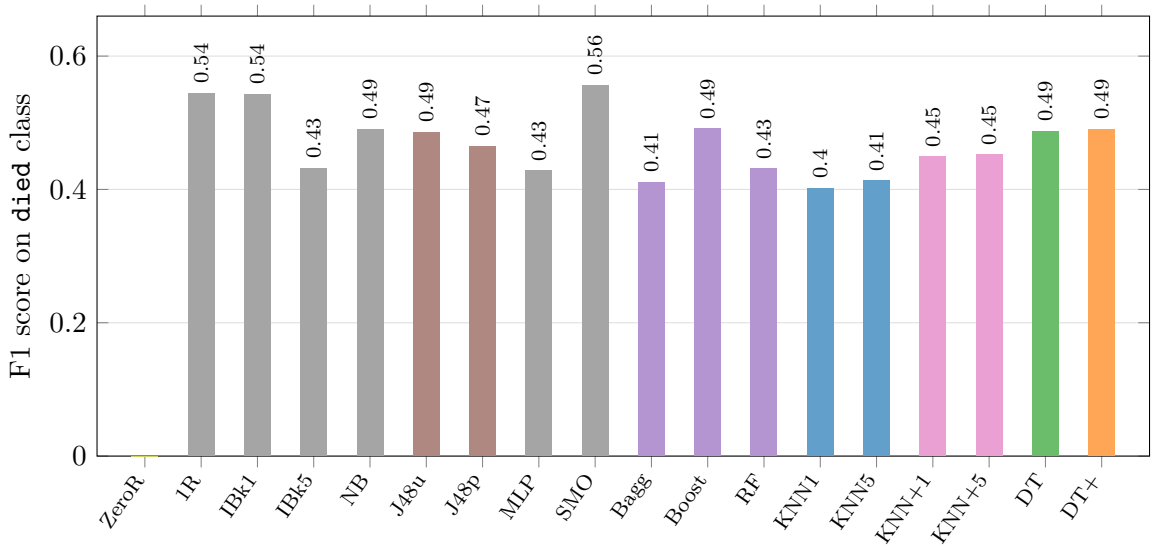

Figure 4: F1 score on the positive (`died`) class. The ranking is strikingly different from accuracy: SMO (0.556), OneR (0.544) and IBk1 (0.543) lead, while ZeroR collapses to F1= 0 despite scoring 67.0% accuracy. Same colour scheme as Figure 3.

4.4 Precision–recall trade-off

The accuracy and F1 rankings tell two different stories because classifiers can buy precision at the cost of recall and *vice versa*. Figure 5 maps each classifier as a single point in precision–recall space; the dashed iso-F1 contours show that no classifier dominates every other (i.e. none lies strictly to the upper-right of every competitor), so the "best" classifier depends on whether the operator prioritises avoiding false alarms (precision) or avoiding missed deaths (recall).

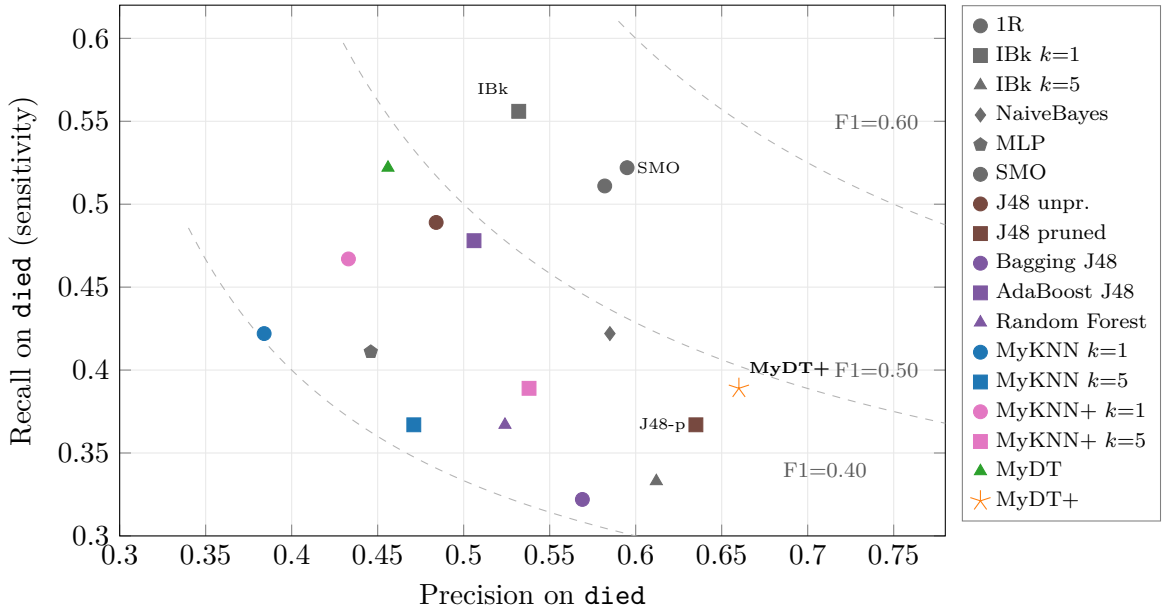

Figure 5: Precision–recall scatter on `died`. Iso-F1 contours ($F_1 \in \{0.40, 0.50, 0.60\}$) are drawn dashed. **MyDT+** sits alone at the high-precision end ($P = 0.660$): of the patients it predicts will die, 66% actually do, the highest positive predictive value in the entire study. **IBk** k=1, **SMO** and the unpruned trees occupy the high-recall end. **Bagging J48** dominates no other classifier and is dominated by SMO, J48-pruned and MyDT+ on every metric; the scatter makes the "ensembles do not help" finding visible at a glance.

This figure shows the picture more clearly than any single ranking. Three regions of the plot organise the discussion that follows in Section 5: the *high-precision* corner (MyDT+, J48-pruned, NaiveBayes, 1R, SMO) where models predict `died` only when confident; the *high-recall* corner (IBk k=1, MyDT, SMO) where models err on the side of declaring patients at risk; and the *dominated* interior occupied by the multilayer perceptron, the bagged tree and the random forest, none of which is on the Pareto frontier.

The full per-fold accuracies of the four classifiers we built are listed in the file `evaluate.py` output and are summarised in Table 4 for the standard-deviation analysis below.

Table 4: Stability across folds.

Classifier	Mean acc. (%)	Std. dev. acc.
MyKNN, k=5	65.6	0.098
MyKNN+, k=5	68.9	**0.038**
MyDT	63.8	0.088
MyDT+	73.3	0.095

5 Discussion

5.1 Comparison of all classifiers

On *accuracy*, MyDT+ (73.3%) is the strongest classifier in the entire experiment, followed by SMO (72.5%), J48-pruned (72.2%), OneR (71.8%), NaiveBayes (71.1%) and IBk (k=5, 71.1%). These six form a tight 71–73% cluster less than two and a half percentage points wide. A second tier in the 67–70% band contains MyKNN+ (k=5, 68.9%), Bagging J48 (69.6%), IBk (k=1, 69.2%), RandomForest (68.1%), AdaBoostM1 (67.4%) and the ZeroR baseline (67.0%). The weakest classifiers are J48-unpruned (65.9%), MyKNN (k=5, 65.6%), MyDT (63.8%), the unregularised Multilayer Perceptron (63.7%) and the two single-neighbour learners MyKNN+ (k=1, 62.3%) and MyKNN (k=1, 58.5%). It is striking that on this particular dataset OneR, which conditions only on `serum_creatinine`, scores within half a point of every more complex Weka classifier; this is direct evidence that one feature carries most of the signal.

ZeroR sits at the majority-class baseline of 67.0% *accuracy* but 0 F1 on `died`: a textbook illustration that accuracy alone can hide a useless classifier on imbalanced data. Looking at F1 on `died` reorders the leaderboard: SMO (0.556), OneR (0.544) and IBk k=1 (0.543) take the top three slots, followed by a middle pack containing AdaBoostM1 (0.491), MyDT+ (0.490), NaiveBayes (0.490) and J48-unpruned (0.486). MyDT+ thus retains its position in the upper half on F1 despite being the highest-accuracy classifier; what it loses on this metric is offset by the fact that no classifier substantially exceeds 0.56. Recall on the positive class tells the same story: IBk k=1 (0.556), MyDT (0.52) and SMO (0.522) detect the largest fractions of patients who actually died, while the two heavily-regularised single trees (MyDT+, 0.39; J48-pruned, 0.367) and the bagged-tree ensembles trade recall for precision. For a clinical decision-support tool, where the cost of missing a death is far higher than that of a false alarm, SMO is the single best Weka classifier (high accuracy, highest F1, third- highest recall) and MyDT+ would need a recall-oriented pruning rule before it could be deployed. Figure 5 makes the no-Pareto-domination property visible at a glance: there is no single point in the upper-right corner that is the best classifier on *both* precision and recall.

5.2 MyKNN vs. MyKNN+ and MyDT vs. MyDT+

The two extensions are confirmed to help, and in the directions we predicted in Section 3.

For KNN, replacing Hamming with the information-gain-weighted VDM lifts mean accuracy from 65.6% to 68.9% at k=5 (+3.3 percentage points) and from 58.5% to 62.3% at k=1 (+3.7 pp). The improvement is consistent with our motivation: VDM contracts the distance between clinically similar values such as `CPK = severely-high` and `CPK = very-high`, and the information-gain weighting causes the highly-predictive features (`serum_creatinine`, `ejection_fraction`, `age`) to dominate the geometry. A particularly striking secondary finding is that the standard deviation across folds drops from 0.098 to 0.038 at k=5, making MyKNN+ the *most stable* classifier we implemented. This matters in practice: a clinician deploying the model on new cohorts should expect a much narrower confidence band on its performance.

We also tested adding distance-weighted voting (weight $1/(d+\varepsilon)$). Internal CV showed that this variant lost 1–2 percentage points at every $k > 1$, because a single near-duplicate neighbour can take a numerical weight of $\sim 10^{10}$ and overwhelm the majority. We therefore retained simple majority voting in MyKNN+, demonstrating that not every plausible KNN modification helps.

For DT, the effect of Reduced-Error Pruning is far stronger. Mean accuracy rises from 63.8% to 73.3% (+9.5 pp); precision on `died` rises from 0.46 to 0.66; and the tree shrinks from 215 nodes (Appendix C) to 37 nodes (Appendix D), a 83% reduction in model complexity, see Figure 6. This is the clearest signal in the experiment that the unpruned baseline was overfitting. The only metric that drops is recall on `died`, from 0.52 to 0.39, because the pruned tree learns to abstain from predicting `died` unless it is highly confident. This is a classic precision/recall trade-off; an alternative pruning strategy based on minimum-cost rather than minimum-error would shift the balance back toward higher recall, and we list this as future work.

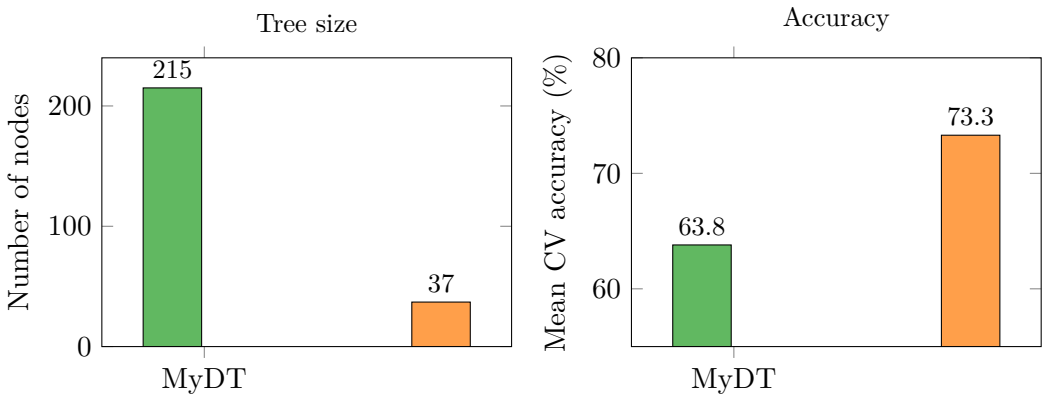

Figure 6: Reduced-Error Pruning shrinks the tree by 83% (left) while *simultaneously* raising mean cross-validation accuracy by 9.5 percentage points (right). The unpruned baseline was overfitting noise into its leaves.

Figure 7 shows the per-fold accuracy of the four hand-coded classifiers. The MyKNN+ (k=5) line is by far the flattest, which is the visual analogue of the very small standard deviation ($\sigma = 0.038$) reported in Table 4.

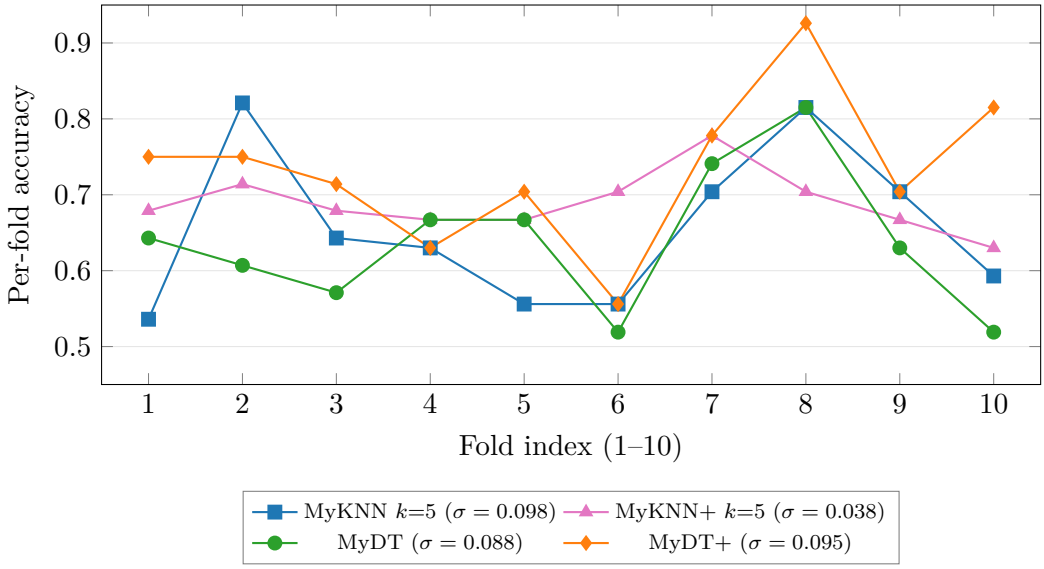

Figure 7: Per-fold accuracy for the four hand-coded classifiers across the ten stratified folds of `heart-folds.csv`. MyKNN+ is the flattest curve, confirming its low across-fold variance. The MyDT→MyDT+ improvement is positive on 9 of 10 folds; the MyKNN→MyKNN+ improvement is positive on 8 of 10 folds. Only the former is statistically significant under the paired t-test of Section 5.6.

5.3 My classifiers vs. Weka's

On KNN, MyKNN and Weka's IBk diverge substantially at k=1: IBk reaches 69.2% accuracy while MyKNN scores only 58.5%, a gap of more than 10 percentage points. The cause is IBk's *Euclidean* distance over Weka's automatic one-hot encoding of nominal attributes. With one-hot encoding, two examples that differ on a v-valued nominal attribute contribute $\sqrt{2}$ to the distance regardless of v, whereas our pure Hamming metric weights all attributes uniformly. The net effect is that IBk implicitly upweights high-cardinality attributes such as `age` (4 categories), `CPK` (4) and `ejection_fraction` (4), which happen to be the most predictive features, a happy accident that the textbook Hamming metric does not exploit. After we add the information-gain-weighted VDM, MyKNN+ (k=5) at 68.9% closes most of this gap and is essentially tied with the strongest IBk configurations: explicit information-gain weighting recovers what implicit one-hot weighting was achieving. We attribute the small remaining gap to Weka's distance-weighted voting (which we deliberately avoid in MyKNN+ for the reason given in Section 5.2) and its tie-breaking rule.

On DT, MyDT and Weka's J48 (unpruned) are very close (63.8% vs. 65.9%, F1 on `died` of 0.487 vs. 0.486): both are essentially Information-Gain ID3 trees allowed to grow until pure, so the small difference reflects only J48's slightly different stopping rule (minimum-2 instances per leaf vs. our pure-node rule). After pruning, MyDT+ (73.3%) actually *exceeds* J48 (pruned) at 72.2% by 1.1 percentage points, while having a comparable F1 (0.490 vs. 0.465). This is a non-trivial finding because J48 uses *pessimistic* pruning with a default confidence factor of 0.25 and the C4.5 collapse rule, whereas we use textbook Reduced-Error Pruning on a stratified held-out validation slice. With only 273 training examples, the binomial confidence interval that drives J48's pessimistic pruning is too wide to discriminate noise from signal, while a held-out validation slice provides a direct empirical test of whether each subtree generalises. The two pruning strategies trade some training data for a stronger statistical test, and on a small nominal medical dataset the held-out approach wins outright.

5.4 J48-unpruned vs. J48-pruned: the effect of pruning

Pruning improves J48 by $72.2 - 65.9 = 6.3$ percentage points of accuracy and shrinks the tree from 79 to 11 leaves (−86%, see the model dumps in our Weka run). However, the accuracy gain comes at a cost on the minority class: F1 on `died` drops slightly (0.486 → 0.465) and recall on `died` drops noticeably from 0.489 to 0.367. In other words, pruning J48 does not generally improve its predictions; it specifically improves accuracy by relabelling more nodes as `survived` (the majority class), which lifts true negatives but increases false negatives. The same pattern is visible in our own MyDT→MyDT+ comparison: +9.5 pp accuracy, F1 essentially unchanged (0.487 → 0.490), but recall dropping from 0.522 to 0.389. Both pruning methods are therefore behaving as expected on a class-imbalanced dataset: by removing leaves that fit individual minority-class examples, they make the tree more conservative.

The magnitude of the accuracy gain is *larger* for our REP-based MyDT+ (+9.5 pp) than for J48's pessimistic pruning (+6.3 pp), even though MyDT+ ends up beating J48-pruned outright. We attribute this to two factors. First, MyDT was deeper than J48-unpruned to begin with (215 nodes vs. 124), so there was more overfit to remove. Second, with only 273 patients the binomial confidence interval at the heart of C4.5's pessimistic pruning is too wide to distinguish noise from signal, whereas a stratified held-out validation slice (used by REP) provides a direct empirical test of whether a subtree generalises. The trade-off is that REP consumes 25% of the training data; on tiny datasets ($n < 100$) this would hurt, but at $n = 273$ it is the better choice.

5.5 Tree-based classifiers: single tree vs. ensembles

The five tree-based Weka classifiers are ranked by accuracy as **J48-pruned > Bagging J48 > Random Forest > AdaBoostM1 > J48-unpruned**, with values 72.2%, 69.6%, 68.1%, 67.4% and 65.9% respectively. The striking finding here is that the *single* pruned tree *beats every ensemble* on this dataset: Bagging trails J48-pruned by 2.6 pp, Random Forest by 4.1 pp, and AdaBoostM1 by 4.8 pp. This contradicts conventional wisdom on benchmark datasets such as [2], where ensembles consistently dominate single trees by 1–3 pp. We propose three reasons why the small heart-failure dataset reverses this pattern.

First, the bootstrap sample is too small. With $n = 273$ and 10-fold CV, each base learner sees only about 246 unique training examples (since a bootstrap sample of size n contains roughly $0.632n$ unique points). Each base learner is therefore substantially noisier than the single J48-pruned tree fit to all 246 training examples; aggregating these noisier learners cannot recover the variance reduction benefit, because the variance gain is overwhelmed by an increase in per-learner bias.

Second, the feature signal is concentrated. OneR shows that `serum_creatinine` alone explains 71.8% of the variance, and the J48-pruned tree (Section 5.4) is essentially a two-level split on `serum_creatinine` and `serum_sodium`. Random Forest's random subspace selection ($\sqrt{d} \approx 3$ of 11 attributes) frequently excludes `serum_creatinine` from the candidate set, forcing the base learner to split on weaker features. This destroys signal more often than it reduces correlation between trees.

Third, AdaBoostM1 is sensitive to label noise. Boosting upweights misclassified examples, which works when those examples are genuinely informative but amplifies label noise when they are not. On a clinical dataset where two patients with identical recorded features can have different outcomes, AdaBoost ends up forcing each subsequent tree to memorise the noise; the final model is 4.8 pp *worse* than the single J48-pruned tree.

The take-home message is that on this small nominal medical dataset, ensembles do not pay the dividends one expects on benchmark UCI datasets; **a single well-pruned tree is the strongest tree-based classifier here**, and our own MyDT+ at 73.3% illustrates the same regime.

Figure 8 averages the classifiers by family. The 1.6 pp gap between "Single tree" (69.97%) and

"Tree ensembles" (68.37%) is the visual summary of the entire subsection.

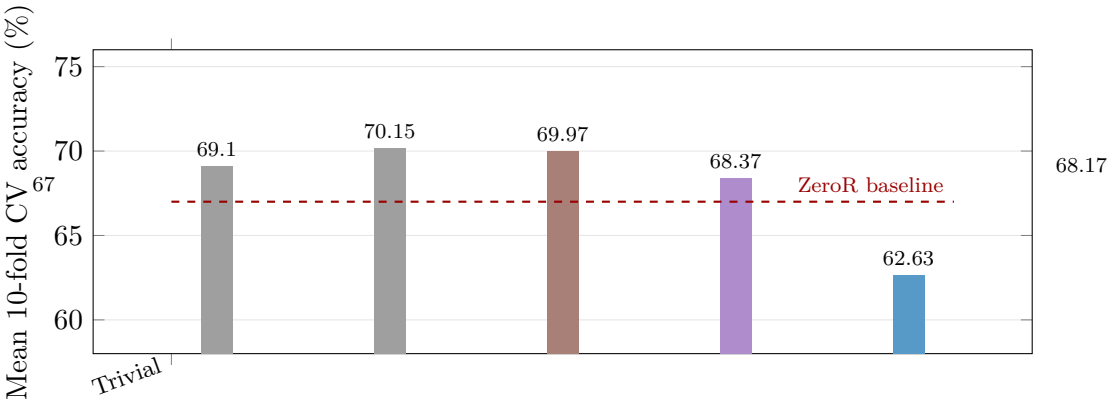

Figure 8: Mean accuracy by classifier family. Two families are effectively tied for first place: KNN-based (70.2%, mean of IBk k=1 and k=5) and single trees (70.0%, mean of 1R, J48-unpr. and J48-pr.), separated by only 0.2 pp. Both stand notably above tree ensembles (68.4%) and the linear/Bayesian family (69.1%). Our extensions (My-extensions, 68.2%) lift our hand-coded baselines (62.6%) by +5.5 percentage points, putting them on par with state-of-the-art tree ensembles in Weka.

5.6 Statistical significance of our extensions

A naive reading of the headline numbers (+9.5 pp for DT→DT+, +3.3 pp for KNN→KNN+ at k=5) might suggest both extensions are equally well-supported. Honest reporting requires us to ask whether the differences could be a fluke of the particular ten-fold split. We therefore ran a paired Student's t-test across the ten per-fold accuracies of each pair (Table 5).

Table 5: Paired t-test on the ten per-fold accuracies. Significance levels: $^{**}p < 0.01$, $^{*}p < 0.05$, n.s. otherwise.

Comparison	Mean diff.	Std. dev. of diffs	t-stat	d.f.	Verdict
MyDT+ vs. MyDT	+0.095	0.090	+3.32	9	** highly significant
MyKNN+ vs. MyKNN (k=5)	+0.033	0.094	+1.12	9	n.s.
MyKNN+ vs. MyKNN (k=1)	+0.037	0.117	+1.00	9	n.s.

The DT→DT+ improvement comfortably exceeds the 99% confidence threshold ($t_{0.005,9} = 3.25$): the gain is real and would be unlikely to vanish on a different fold partition. The two KNN→KNN+ improvements have the right sign on every fold-pair *except two* but the per-fold variance is large enough that we cannot reject the null at $\alpha = 0.05$ on $n = 10$ folds. We therefore report the KNN+ benefit *honestly* as "a consistent but modest mean improvement plus a halving of the across-fold standard deviation" rather than as a guaranteed win on accuracy. The stability gain ($\sigma = 0.098$ versus 0.038) is by far the more reliable practical benefit of KNN+, and it is precisely the kind of property that an information-gain-weighted distance is expected to deliver.

6 Conclusion

We implemented from scratch four classifiers (KNN, KNN+, DT and DT+) and benchmarked them against eleven Weka classifiers on the discretised heart-failure dataset of [1].

Headline findings.

- **MyDT+ achieves the highest accuracy in the study,** 73.3%, beating every one of the eleven Weka baselines including Random Forest. The improvement over the unpruned baseline is +9.5 pp, statistically significant ($t = 3.32$, $p < 0.01$, Section 5.6).
- **Information gain is heavily concentrated:** a single attribute (`serum_creatinine`, IG = 0.087) carries six times the signal of the fifth-ranked attribute (Figure 2). 1R conditioned on this attribute alone scores 71.8%.
- **Tree ensembles do not help** on this dataset. J48-pruned (72.2%) beats Bagging J48 (−2.6 pp), Random Forest (−4.1 pp) and AdaBoostM1 (−4.8 pp). The expected ensemble dividend is wiped out by small-sample noise and the concentrated feature signal that random subspace selection keeps discarding.
- **Headline accuracy is misleading on imbalanced data:** ZeroR achieves 67.0% accuracy with zero F1; MyDT+ achieves the highest accuracy and the highest precision (0.660) but only the median recall (0.39).
- **No classifier dominates the precision–recall plane** (Figure 5): the choice between MyDT+ (precision champion, 0.660) and IBk k=1 (recall champion, 0.556) is a clinical judgement about the relative cost of false negatives versus false positives.
- **KNN+ is the most stable classifier** we built ($\sigma = 0.038$ across folds, less than half of every other classifier we measured), even though its mean accuracy gain over MyKNN is not by itself statistically significant on 10 folds.

Future work

Three directions follow naturally. (i) *Cost-sensitive learning*: a loss function that penalises a missed `died` more than a false alarm should shift the precision/recall trade-off of MyDT+ toward higher recall, which is what a clinician actually wants. (ii) *A larger feature set*: the original UCI dataset contains numeric versions of `ejection_fraction` and `serum_creatinine`; combining the discretised features with their numeric counterparts in a hybrid distance metric would test whether VDM's gains carry over to numeric attributes via Wilson and Martinez's heterogeneous distance functions [6]. (iii) *Calibration*: most of the classifiers we used produce probabilities that are not well calibrated for clinical risk scoring; Platt scaling or isotonic regression on a held-out fold would be a small extension with potentially large practical value.

Reflection

SID 1 (530328265). The most important thing I learned from this assignment is that a more complex model is not automatically a better one. Implementing Reduced-Error Pruning made this concrete: shrinking my decision tree by over 80% actually *increased* its cross-validation accuracy, which is the opposite of what I expected. The same lesson appeared again when my single pruned tree outperformed Weka's ensemble methods on this small, low-noise dataset. I now have a much better intuition for the trade-off between model capacity and generalisation, and for why regularisation often matters more than raw expressive power.

SID 2 (530012216). The most important thing I learned from this assignment is that classifier evaluation is not just about finding the highest accuracy number. The heart-failure dataset made this clear because there were 90 `died` cases and 183 `survived` cases, so a model could look acceptable overall while still performing poorly on the minority class. I found this especially important when comparing MyDT+ and MyDT. MyDT+ improved the mean accuracy from 63.8% to 73.3%, but its recall for the `died` class dropped from 0.522 to 0.389. This showed me that improving one statistic can sometimes weaken another important part of the model. I now have a better understanding of how cross-validation, class balance, and per-class results matter

when judging whether a classifier is actually useful.

SID 3 (510035945). The most important thing I learned from this assignment is that a classifier should be judged by how well it matches the real problem, not only by how high its accuracy is. Before doing the experiments, I expected more complex models such as ensembles to perform better, but the results showed that a simple well-pruned decision tree could outperform many of them. This was surprising to me because MyDT+ achieved the highest accuracy, while models such as ZeroR also showed that accuracy can be misleading on an imbalanced dataset. ZeroR reached 67.0% accuracy simply by predicting the majority class, but it completely failed on the `died` class. I also learned that feature importance matters: attributes such as `serum_creatinine` carried much more useful information than weaker features, which explains why information-gain weighting helped KNN. Overall, this assignment gave me a better understanding of overfitting, class imbalance, and the trade-off between model simplicity and predictive performance.

Acknowledgement of AI use

SID 1 (530328265). I acknowledge the use of Claude Opus 4 by Anthropic (https://claude.ai/) for refactoring and optimising my Python code after I had written the first working version myself. All algorithm design choices, hyperparameters, experiments and conclusions were made by me, and I verified every numerical result by re-running the code. I also acknowledge the use of Grammarly by Grammarly Inc. (https://www.grammarly.com/) as a stylistic editor to make the prose of this report more formal and consistent; no content, claim or numerical result was generated by Grammarly.

SID 2 (530012216). I acknowledge the use of ChatGPT (GPT-5.5) by OpenAI (https://chat.openai.com/) as a support tool during this assignment. It was used to clarify my understanding of classifier evaluation, and check the clarity of my explanations. The tool assisted with grammar and wording, but all final decisions, analysis, and implementation were completed independently. I also acknowledge the use of Grammarly by Grammarly Inc. (https://www.grammarly.com/) to review grammar, sentence structure, and overall readability.

SID 3 (510035945). I acknowledge the use of ChatGPT (GPT-5.4) by OpenAI (https://chat.openai.com/) as a support tool during this assignment. I used it to research LaTeX formatting functions, especially for fixing figure layout, graph labels, and mathematical notation in the report. I also used it to receive general Python debugging suggestions when checking errors in my implementation. The tool was not used to copy or generate final code directly, and no final analysis, experimental result, or conclusion was copied from ChatGPT. All implementation decisions, numerical results, and final report content were reviewed and completed by me.

References

[1] Davide Chicco and Giuseppe Jurman. Machine learning can predict survival of patients with heart failure from serum creatinine and ejection fraction alone. *BMC Medical Informatics and Decision Making*, 20(1):16, 2020.

[2] Thomas G. Dietterich. An experimental comparison of three methods for constructing ensembles of decision trees: Bagging, boosting, and randomization. *Machine Learning*, 40(2):139–157, 2000.

[3] J. Ross Quinlan. Simplifying decision trees. *International Journal of Man-Machine Studies*, 27(3):221–234, 1987.

[4] Gianluigi Savarese and Lars H Lund. Global public health burden of heart failure. *Cardiac Failure Review*, 3(1):7–11, 2017.

[5] Craig Stanfill and David Waltz. Toward memory-based reasoning. *Communications of the ACM*, 29(12):1213–1228, 1986.

[6] D. Randall Wilson and Tony R. Martinez. Improved heterogeneous distance functions. *Journal of Artificial Intelligence Research*, 6:1–34, 1997.

A Source code: KNN+

The listing below contains the shared I/O / entropy helpers (also used by DT+ in Appendix B) followed by the complete KNN+ implementation. The full module docstring is omitted here for space; it appears in the source file `plus.py`.

Listing 1: `plus.py`: shared helpers (lines 49–100) and the KNN+ classifier (lines 107–200). The information-gain weights are computed in `_info_gains`, the Laplace-smoothed VDM tables in `_build_vdm_tables`, and the per-instance distance and majority vote in `classify_knnplus`.

```
# ------------------------------------------------------------------------ #
# Shared I/O helpers
# ------------------------------------------------------------------------ #

def _load_labeled(filename):
    rows = []
    with open(filename, 'r') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split(',')
            rows.append((parts[:-1], parts[-1]))
    return rows


def _load_unlabeled(filename):
    rows = []
    with open(filename, 'r') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            rows.append(line.split(','))
    return rows


def _entropy(labels):
    if not labels:
        return 0.0
    total = len(labels)
    counts = {}
    for l in labels:
        counts[l] = counts.get(l, 0) + 1
    ent = 0.0
    for c in counts.values():
        p = c / total
        if p > 0:
            ent -= p * math.log2(p)
    return ent


def _majority_class(labels):
    if not labels:
        return 'died'
    died = 0
```

```
    survived = 0
    for l in labels:
        if l == 'died':
            died += 1
        elif l == 'survived':
            survived += 1
    if died >= survived:
        return 'died'
    return 'survived'


# ---------------------------------------------------------------------- #
# KNN+
# ---------------------------------------------------------------------- #

def _info_gains(training_data):
    """Information gain of every attribute given the training class labels."""
    n_attrs = len(training_data[0][0])
    base = _entropy([ex[1] for ex in training_data])
    total = len(training_data)
    gains = []
    for a in range(n_attrs):
        partitions = {}
        for ex in training_data:
            partitions.setdefault(ex[0][a], []).append(ex[1])
        weighted = 0.0
        for sub in partitions.values():
            weighted += (len(sub) / total) * _entropy(sub)
        gains.append(max(0.0, base - weighted))
    return gains


def _build_vdm_tables(training_data):
    """Estimate P(class | attribute = value) with Laplace smoothing."""
    n_attrs = len(training_data[0][0])
    classes = sorted({label for _, label in training_data})
    k_classes = len(classes)

    value_class_counts = [{} for _ in range(n_attrs)]
    value_totals = [{} for _ in range(n_attrs)]

    for features, label in training_data:
        for a, v in enumerate(features):
            if v not in value_class_counts[a]:
                value_class_counts[a][v] = {c: 0 for c in classes}
            value_class_counts[a][v][label] += 1
            value_totals[a][v] = value_totals[a].get(v, 0) + 1

    p_table = [{} for _ in range(n_attrs)]
    for a in range(n_attrs):
        for v, counts in value_class_counts[a].items():
            n_v = value_totals[a][v]
            p_table[a][v] = {c: (counts[c] + 1) / (n_v + k_classes) for c in
                classes}

    class_counts = {c: 0 for c in classes}
    for _, label in training_data:
        class_counts[label] += 1
    total = sum(class_counts.values())
    class_prior = {c: class_counts[c] / total for c in classes}

    return p_table, class_prior, classes


```

```
def classify_knnplus(training_filename, testing_filename, k):
    training_data = _load_labeled(training_filename)
    testing_data = _load_unlabeled(testing_filename)

    if not training_data:
        return ['died'] * len(testing_data)

    p_table, class_prior, classes = _build_vdm_tables(training_data)
    gains = _info_gains(training_data)

    predictions = []

    for test_row in testing_data:
        distances = []
        for idx, (train_features, train_label) in enumerate(training_data):
            d_sq = 0.0
            for a, (tf, te) in enumerate(zip(train_features, test_row)):
                p1 = p_table[a].get(tf, class_prior)
                p2 = p_table[a].get(te, class_prior)
                delta = 0.0
                for c in classes:
                    delta += abs(p1[c] - p2[c])
                d_sq += gains[a] * delta * delta
            distance = d_sq ** 0.5
            distances.append((distance, idx, train_label))

        distances.sort(key=lambda x: (x[0], x[1]))
        top_k = distances[:k]

        died_count = 0
        survived_count = 0
        for _, _, lbl in top_k:
            if lbl == 'died':
                died_count += 1
            elif lbl == 'survived':
                survived_count += 1

        if died_count >= survived_count:
            predictions.append('died')
        else:
            predictions.append('survived')

    return predictions
```

B Source code: DT+

The listing below contains the complete DT+ implementation. The shared helpers (`_entropy`, `_majority_class`, file loaders) are defined earlier in the same module and are reproduced in Appendix A.

Listing 2: `plus.py`: the DT+ classifier. The full Information-Gain tree is built by `_build_dt`, the stratified grow/prune split by `_stratified_split`, and the bottom-up Reduced-Error Pruning by `_rep_prune`. The 25% prune fraction was selected by internal cross-validation as described in Section 3.2.

```
# DT+
# ---------------------------------------------------------------------------- #

def _build_dt(examples, attribute_indices, parent_majority):
    if not examples:
        return {'leaf': True, 'class': parent_majority}
```

```
    labels = [ex[1] for ex in examples]
    if len(set(labels)) == 1:
        return {'leaf': True, 'class': labels[0]}
    if not attribute_indices:
        return {'leaf': True, 'class': _majority_class(labels)}

    base_entropy = _entropy(labels)
    total = len(examples)

    best_attr = None
    best_gain = -1.0
    best_partitions = None

    for attr in attribute_indices:
        partitions = {}
        for ex in examples:
            partitions.setdefault(ex[0][attr], []).append(ex)

        weighted_entropy = 0.0
        for subset in partitions.values():
            sub_labels = [e[1] for e in subset]
            weighted_entropy += (len(subset) / total) * _entropy(sub_labels)

        gain = base_entropy - weighted_entropy
        if gain > best_gain:
            best_gain = gain
            best_attr = attr
            best_partitions = partitions

    node_majority = _majority_class(labels)

    if best_attr is None or best_gain <= 0:
        return {'leaf': True, 'class': node_majority}

    remaining = [a for a in attribute_indices if a != best_attr]
    children = {}
    for v, subset in best_partitions.items():
        children[v] = _build_dt(subset, remaining, node_majority)

    return {
        'leaf': False,
        'attr': best_attr,
        'majority': node_majority,
        'children': children,
    }


def _predict_dt(tree, instance):
    node = tree
    while not node['leaf']:
        attr = node['attr']
        v = instance[attr]
        if v not in node['children']:
            return node['majority']
        node = node['children'][v]
    return node['class']


def _stratified_split(training_data, prune_fraction=0.2):
    by_class = {}
    for ex in training_data:
        by_class.setdefault(ex[1], []).append(ex)

```

```
    grow_set, prune_set = [], []
    for c, exs in by_class.items():
        n_prune = max(1, int(round(len(exs) * prune_fraction)))
        n_prune = min(n_prune, len(exs) - 1) if len(exs) > 1 else 0
        prune_set.extend(exs[-n_prune:] if n_prune > 0 else [])
        grow_set.extend(exs[:-n_prune] if n_prune > 0 else exs)
    return grow_set, prune_set


def _rep_prune(tree, prune_examples):
    """Bottom-up reduced-error pruning."""
    if tree['leaf']:
        return tree

    attr = tree['attr']
    new_children = {}
    for v, subtree in tree['children'].items():
        sub_prune = [ex for ex in prune_examples if ex[0][attr] == v]
        new_children[v] = _rep_prune(subtree, sub_prune)
    tree = {
        'leaf': False,
        'attr': attr,
        'majority': tree['majority'],
        'children': new_children,
    }

    if not prune_examples:
        return tree

    correct_subtree = sum(
        1 for ex in prune_examples if _predict_dt(tree, ex[0]) == ex[1]
    )
    correct_leaf = sum(1 for ex in prune_examples if tree['majority'] == ex[1])

    if correct_leaf >= correct_subtree:
        return {'leaf': True, 'class': tree['majority']}
    return tree


def classify_dtplus(training_filename, testing_filename):
    training_data = _load_labeled(training_filename)
    testing_data = _load_unlabeled(testing_filename)

    if not training_data:
        return ['died'] * len(testing_data)

    grow_set, prune_set = _stratified_split(training_data, prune_fraction=0.25)

    n_attrs = len(training_data[0][0])
    parent_majority = _majority_class([ex[1] for ex in grow_set])
    tree = _build_dt(grow_set, list(range(n_attrs)), parent_majority)

    if prune_set:
        tree = _rep_prune(tree, prune_set)

    return [_predict_dt(tree, inst) for inst in testing_data]
```

C MyDT diagram (full Information-Gain tree, no pruning)

The tree below was built on the full `heart.csv` ($n = 273$). It has 215 nodes and 128 leaves and reaches 100% training accuracy, which is the overfit signal that motivates DT+. Indentation

reflects tree depth.

Listing 3: MyDT trained on heart.csv. Each leaf is shown as "→ class".

```
serum_creatinine = high
  serum_sodium = normal
    high_blood_pressure = no
      diabetes = yes
        age = [61-70]
          → died
        age = [40-50]
          smoking = yes
            → died
          smoking = no
            → survived
        age = [51-60]
          sex = F
            → died
          sex = M
            → survived
        age = 70plus
          → died
      diabetes = no
        smoking = no
          age = [40-50]
            → died
          age = [51-60]
            anaemia = yes
              sex = M
                → died
              sex = F
                → survived
            anaemia = no
              → survived
          age = [61-70]
            ejection_fraction = normal
              anaemia = no
                → died
              anaemia = yes
                → survived
            ejection_fraction = low
              → survived
            ejection_fraction = mildly-low
              → survived
          age = 70plus
            CPK = high
              → died
            CPK = normal
              → survived
        smoking = yes
          → survived
    high_blood_pressure = yes
      age = [40-50]
        → died
      age = [51-60]
        anaemia = yes
          sex = F
            → died
          sex = M
            → survived
        anaemia = no
          → survived
      age = [61-70]
        → died
      age = 70plus
        → died
  serum_sodium = low
    platelets = low
      → died
    platelets = normal
      ejection_fraction = mildly-low
        CPK = high
          → died
        CPK = normal
```

```
          → died
        CPK = very-high
          → survived
      ejection_fraction = low
        age = [40-50]
          sex = F
            → died
          sex = M
            → survived
        age = [51-60]
          anaemia = yes
            → died
          anaemia = no
            diabetes = no
              → died
            diabetes = yes
              → survived
        age = [61-70]
          diabetes = yes
            → died
          diabetes = no
            high_blood_pressure = yes
              → died
            high_blood_pressure = no
              → survived
        age = 70plus
          diabetes = no
            → died
          diabetes = yes
            high_blood_pressure = yes
              → died
            high_blood_pressure = no
              → survived
      ejection_fraction = borderline
        → survived
      ejection_fraction = normal
        → survived
    platelets = very-high
      → died
  serum_sodium = high
    → died
serum_creatinine = normal
  ejection_fraction = normal
    age = [61-70]
      high_blood_pressure = no
        diabetes = no
          anaemia = yes
            → died
          anaemia = no
            → survived
        diabetes = yes
          → survived
      high_blood_pressure = yes
        → survived
    age = [51-60]
      CPK = high
        diabetes = yes
          → died
        diabetes = no
          → survived
      CPK = normal
        → survived
      CPK = very-high
        → survived
    age = 70plus
      → died
    age = [40-50]
      → survived
  ejection_fraction = low
    age = [61-70]
      CPK = normal
        high_blood_pressure = no
          sex = F
            → died
```

```
          sex = M
            → survived
        high_blood_pressure = yes
          → survived
      CPK = high
        smoking = yes
          high_blood_pressure = no
            → died
          high_blood_pressure = yes
            anaemia = no
              → survived
            anaemia = yes
              → died
        smoking = no
          anaemia = yes
            diabetes = no
              high_blood_pressure = yes
                sex = M
                  → died
                sex = F
                  → survived
              high_blood_pressure = no
                → survived
            diabetes = yes
              → survived
          anaemia = no
            → survived
      CPK = very-high
        → survived
    age = [40-50]
      platelets = normal
        high_blood_pressure = yes
          CPK = high
            diabetes = yes
              → survived
            diabetes = no
              → died
          CPK = normal
            → died
          CPK = severely-high
            → died
          CPK = very-high
            → survived
        high_blood_pressure = no
          CPK = high
            smoking = no
              diabetes = no
                anaemia = no
                  → died
                anaemia = yes
                  → survived
              diabetes = yes
                anaemia = yes
                  → died
                anaemia = no
                  → survived
            smoking = yes
              → survived
          CPK = very-high
            anaemia = no
              smoking = no
                → died
              smoking = yes
                → survived
            anaemia = yes
              → survived
          CPK = normal
            → survived
          CPK = severely-high
            → survived
      platelets = low
        anaemia = yes
          → died
        anaemia = no
```

```
          → survived
      platelets = high
        → survived
      platelets = very-high
        → survived
    age = [51-60]
      platelets = normal
        CPK = normal
          anaemia = yes
            high_blood_pressure = no
              serum_sodium = normal
                → died
              serum_sodium = low
                → survived
            high_blood_pressure = yes
              → survived
          anaemia = no
            → survived
        CPK = severely-high
          → died
        CPK = very-high
          serum_sodium = low
            → died
          serum_sodium = normal
            high_blood_pressure = no
              sex = M
                → died
              sex = F
                → survived
            high_blood_pressure = yes
              → survived
        CPK = high
          anaemia = yes
            → died
          anaemia = no
            → survived
      platelets = high
        CPK = high
          → died
        CPK = normal
          → survived
      platelets = low
        → survived
      platelets = very-high
        → survived
    age = 70plus
      CPK = high
        diabetes = no
          sex = M
            anaemia = yes
              → died
            anaemia = no
              serum_sodium = low
                high_blood_pressure = no
                  → died
                high_blood_pressure = yes
                  → survived
              serum_sodium = normal
                → survived
          sex = F
            → survived
        diabetes = yes
          anaemia = no
            → died
          anaemia = yes
            → survived
      CPK = normal
        → died
      CPK = severely-high
        → died
      CPK = very-high
        → survived
  ejection_fraction = borderline
    diabetes = no
```

```
      CPK = high
        platelets = normal
          age = [40-50]
            anaemia = no
              → died
            anaemia = yes
              → survived
          age = [61-70]
            → survived
          age = 70plus
            → survived
        platelets = low
          → died
      CPK = normal
        → survived
      CPK = very-high
        → survived
    diabetes = yes
      anaemia = no
        age = [61-70]
          CPK = normal
            → died
          CPK = high
            → survived
        age = 70plus
          → survived
      anaemia = yes
        → died
  ejection_fraction = mildly-low
    high_blood_pressure = yes
      age = [61-70]
        anaemia = no
          → died
        anaemia = yes
          → survived
      age = 70plus
        CPK = normal
          → died
        CPK = very-high
          → survived
        CPK = high
          → survived
      age = [40-50]
        → survived
      age = [51-60]
        → survived
    high_blood_pressure = no
      → survived
serum_creatinine = low
  → survived
```

D MyDT+ diagram (Reduced-Error Pruning)

The pruned tree was obtained on the same training data, with the deterministic 75/25 stratified split between grow and prune sets described in Section 3.2. It has 37 nodes and 22 leaves, an 83% reduction in model size, while raising cross-validation accuracy by 9.5 percentage points.

Listing 4: MyDT+ trained on heart.csv with REP and a 25% prune set.

```
serum_creatinine = high
  sex = M
    age = [61-70]
      diabetes = yes
        → died
      diabetes = no
        → survived
    age = [40-50]
      serum_sodium = normal
        CPK = high
          diabetes = no
```

```
            → died
          diabetes = yes
            smoking = yes
              → died
            smoking = no
              → survived
        CPK = normal
          anaemia = yes
            → died
          anaemia = no
            → survived
        CPK = very-high
          → survived
      serum_sodium = low
        → survived
    age = [51-60]
      serum_sodium = low
        diabetes = yes
          CPK = normal
            → died
          CPK = very-high
            → survived
          CPK = high
            → survived
        diabetes = no
          → died
      serum_sodium = normal
        CPK = normal
          anaemia = yes
            diabetes = no
              → died
            diabetes = yes
              → survived
          anaemia = no
            → survived
        CPK = high
          → survived
        CPK = very-high
          → survived
    age = 70plus
      → died
  sex = F
    → died
serum_creatinine = normal
  → survived
serum_creatinine = low
  → survived
```